UAV-Assisted Mobile Edge Computing With Digital Twin

Luteng Qiao, *Member, IEEE*, Yong Zhou, *Senior Member, IEEE*, Yuanming Shi, *Senior Member, IEEE*, Sheng Wu, and Tom H. Luan, *Senior Member, IEEE*

***Abstract*—Unmanned aerial vehicles (UAVs)-assisted mobile edge computing (MEC) is a promising technology for providing timely and seamless computing services to mobile users by taking advantage of the flexible mobility. In this paper, we consider a UAV-assisted MEC with digital twin, which effectively guides task-offloading by constructing a virtual representation to monitor real-time changes of the physical MEC system. Considering that the computing tasks arrive more frequently than the UAV makes flight decisions, we propose a two-timescale optimization framework, where the UAV's trajectory is planned on a large timescale, while the offloading decision and computing resource allocation are jointly optimized on a small timescale. With an objective to minimize the system delay while accounting for the constrained computing resources and energy of the UAV, we formulate a two-timescale Markov decision process, followed by developing a customized proximal policy optimization algorithm. To guarantee that the computing resource constraints are always satisfied, we further propose a safe projection method that directly maps the output of the policy network to feasible actions. The convergence and superiority of the proposed algorithm are verified through extensive simulations.**

***Index Terms*—Mobile edge computing, digital twin, unmanned aerial vehicle, Markov decision process.**

I. Introduction

With the rapid growth of real-time intelligent applications, the demand for computing power at the network edge is on the rise. With constrained computing power and limited battery capacity of edge devices, supporting computation-intensive tasks imposes significant challenges on local processing. To alleviate the computing burden on edge devices, mobile edge computing (MEC) emerges as a new technique that offers computing power by deploying servers at the network edge [1]. Specifically, by offloading tasks from edge devices to the edge servers, the computing delay of tasks and energy consumption of edge devices can be significantly reduced. Meanwhile, compared to the cloud computing, the transmission delay can be significantly reduced by deploying servers at the network edge.

Due to time-varying channel conditions, offloading decisions should be adaptively optimized to reduce the latency of MEC [2], [3], [4], [5], [6], [7]. Binary offloading and partial offloading are two typical offloading models in MEC. For binary offloading, each task can be either performed locally or entirely offloaded to the edge server [2], [3], [4], [5]. On the other hand, partial offloading allows a fraction of the task to be offloaded to the edge server for computation [6], [7]. In [3], the authors minimize the latency of multi-user time-division MEC under two offloading models, where a closed-form solution is provided. In [7], the authors propose a hierarchical MEC architecture and minimize the latency by jointly optimizing relay selection, bandwidth allocation, and partial offloading decisions. For the edge-cloud cooperative MEC network, the latency is minimized in [4] under binary offloading models while considering queuing stability. Recently, some advanced communication technologies are integrated into MEC to further reduce its latency, such as non-orthogonal multiple access (NOMA) [8], multi-hop task relaying [9], and mobile edge caching [10], with the offloading decisions being optimized. However, since the users are mobile and the computing tasks are randomly arrived, providing seamless computing services becomes challenging, especially when the MEC servers are fixed at specific locations.

Unmanned aerial vehicles (UAVs) can provide timely and seamless MEC services to mobile users due to their high flexibility. The UAVs can serve either as relays to reduce the transmission delay [11], [12] or as edge servers by providing computing resources [13], [14]. To improve communication security for maritime MEC, the secure computing capacity is minimized in [11]. For the scenarios where the UAV can flexibly switch between half- and full-duplex modes, the authors in [12] propose an effective greedy algorithm to tackle the offloading problem. The authors in [13] study the energy efficiency problem for UAV-assisted MEC while considering service fairness. As the task completion time is unknown in certain scenarios, the path discretization method is adopted to minimize the energy of UAV-assisted MEC in [14]. However, these works typically assume that all time-varying parameters of the MEC system are known, which may not hold in practical scenarios.

Received 5 July 2024; revised 31 December 2024 and 15 March 2025; accepted 2 June 2025. Date of publication 22 October 2025; date of current version 20 November 2025. The work of Yong Zhou was supported by the Natural Science Foundation of Shanghai under Grant 23ZR1442800. The work of Yuanming Shi was supported by the National Natural Science Foundation of China under Grant 62271318. The review of this article was coordinated by Dr. Lin X Cai. *(Corresponding author: Yong Zhou.)*

Luteng Qiao, Yong Zhou, and Yuanming Shi are with the School of Information Science and Technology, ShanghaiTech University, Shanghai 201210, China (e-mail: qiaolt@shanghaitech.edu.cn; zhouyong@shanghaitech.edu.cn; shiym@shanghaitech.edu.cn).

Sheng Wu is with the Key Laboratory of Universal Wireless Communications Ministry of Education, School of Information and Communication Engineering, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: thuraya@bupt.edu.cn).

Tom H. Luan is with the School of Cyber Science and Engineering, Xi'an Jiaotong University, Xi'an 710049, China (e-mail: tom.luan@xjtu.edu.cn).

Digital Object Identifier 10.1109/TVT.2025.3578376

0018-9545 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:07:31 UTC from IEEE Xplore. Restrictions apply.

As many optimization problems are long-term and the system parameters (e.g., task arrival, channel conditions, and user positions) are dynamically changed, it is necessary to obtain real-time system state information. Therefore, the concept of digital twin (DT) is introduced [15]. Specifically, DT provides an effective way to simulate and depict the physical world with a virtual representation and becomes a promising technology for guiding task offloading and resource allocation. Many studies on MEC utilize DT to synchronize the real-time changes of the system state and provide enhanced services [16], [17]. In [16], the authors establish a DT edge network for multi-cell MEC to help reduce the communication cost. To enhance user service satisfaction, the authors consider both static and dynamic DT placement strategies for MEC in [17]. Considering that DT may be inaccurate in some cases, several methods are applied to correct the DT [18], [19], [20], [21]. In [18] and [19], the authors estimate the deviation between the true state and the DT, followed by using this deviation to correct the DT. In [20] and [21], the authors apply a Kalman filter to the DT, for effectively estimating system states from noisy information. The aforementioned works motivate us to leverage the superiority of DT to enhance the performance of UAV-assisted MEC.

Recently, works [22], [23], [24], [25], [26], [27], [28] adopt deep reinforcement learning (DRL) to deal with the long-term optimization problems in MEC by modeling them as Markov decision processes (MDPs). However, the conventional DRL algorithms are designed for unconstrained problems. To address this issue, works [22], [23], [24], [25] deal with the constraints by punishing the infeasible actions, while works [26], [27], [28] project the actions to feasible actions. However, the punishing method (i.e., Lagrangian relaxation method) cannot ensure that the constraints are always satisfied, especially in the initial stage of the training process, as it relies on the values of the state-action value function and the state value function, which cannot be well estimated by neural networks in the initial stage of the training process. Moreover, the punishing method also requires selecting an appropriate penalty value, as selecting an inappropriate penalty value can lead to constraint violations [29]. In contrast, the projection-based method can ensure that the constraints are always satisfied during the training process.

A. Contributions

In this paper, we study a UAV-assisted MEC framework with DT, where the arrival of computing tasks is on a smaller timescale than the flight decisions made by the UAV. We aim to minimize the system delay under the constraints of limited computing and energy resources, via jointly optimizing the UAV's trajectory, computing resource allocation, and offloading decisions. The main challenges of accomplishing such a goal are as follows. First, as the optimization problem involves two different timescales, the conventional DRL algorithms involving one timescale cannot be directly applied to solve the problem. Second, as the Lagrangian relaxation method, which penalizes actions that do not satisfy the constraints, cannot guarantee the computing resource constraints to be always satisfied during the training process, it is necessary to propose a novel method to handle the computing resource constraints. To tackle these challenges, we design a two-timescale DRL algorithm and propose a safe projection method that can guarantee the computing resource constraints to be always satisfied during the training process. The main contributions of this paper are summarized as follows:

- We study a UAV-assisted MEC framework with DT, which facilitates effective task offloading by leveraging the synchronized system state information from the physical world. With an objective to minimize the system delay, we formulate a two-timescale MDP, where the UAV's trajectory is planned on a large timescale, while the offloading decision and computing resource allocation are jointly optimized on a small timescale.
- We propose a customized two-timescale proximal policy optimization (PPO) algorithm to jointly optimize the UAV's trajectory, computing resource allocation, and offloading decisions. To handle the computing resource constraints, we develop a safe projection method that maps the policy network's outputs to feasible actions. The proposed safe projection method ensures that the computing resource constraints are always satisfied during the training process, and the complexity analysis is provided.
- The convergence and superiority of the proposed algorithm are validated through extensive simulations. Results illustrate that the proposed algorithm achieves superior delay performance compared to the baselines under different system parameters, and also verify the superiority of leveraging DT.

The rest of this paper is organized as follows. Section II presents the UAV-assisted MEC network model and formulates the delay minimization problem. Section III transforms the problem into a two-timescale MDP. Section IV presents the proposed customized two-timescale PPO algorithm and safe projection algorithm. Section V provides the simulation results. Section VI concludes the paper.

II. System Model

Consider a UAV-assisted MEC network with DT, which consists of N mobile users and a UAV, as shown in Fig. 1. The UAV equipped with computing resources is deployed to provide computing services for mobile users that typically have limited computation capability. As the computing tasks generally arrive frequently, the UAV makes flight decisions (i.e., speed and angle) on a larger timescale to avoid unnecessary decision-making every time a task arrives, thereby reducing the computation complexity and energy consumption caused by excessively frequent flight decision-making [30], [31]. For instance, each mobile user may generate one video analytic task every few tens of milliseconds [32], which is typically too short a time interval for the UAV to make flight decisions. Therefore, we consider a two-timescale MEC framework consisting of large-timescale UAV trajectory design and small-timescale task offloading. Time is segmented into frames with equal duration, with each frame comprising T time slots. The UAV adjusts its position according to the movement of mobile users on the large timescale (i.e.,

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:07:31 UTC from IEEE Xplore. Restrictions apply.

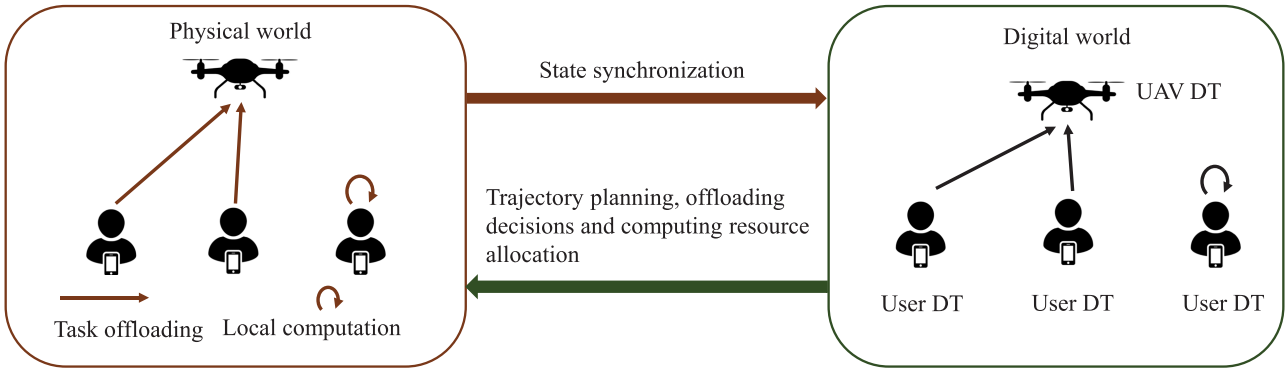


Fig. 1. Illustration of the UAV-assisted MEC with DT.

frame) and allocates the computing resources to mobile users on the small timescale (i.e., time slot).

A. UAV Movement Model

The UAV's 3D coordinates at time slot t are represented as $(x_0(l,t), y_0(l,t), H)$, where (l,t) denotes the t-th time slot of frame l, $x_0(l,t)$ denotes the x-coordinate, $y_0(l,t)$ denotes the y-coordinate, and H denotes the height. To reduce the transmission delay of offloading, the UAV adjusts its coordinates according to the locations of mobile users by determining the flight speed and direction at the beginning of each frame, and then flies in the frame accordingly. Therefore, the x- and y-coordinates of the UAV at time slot $t+1$ are denoted as

$$\begin{aligned} x_0(l,t+1) &= x_0(l,t) + v(l)\cos\alpha(l)\tau, \\ y_0(l,t+1) &= y_0(l,t) + v(l)\sin\alpha(l)\tau, \end{aligned} \tag{1}$$

where τ is the duration of each time slot, and $v(l)$ and $\alpha(l)$ respectively represent the flight speed and angle of the UAV in frame l. The UAV is restricted to fly within a predetermined square area of $[X_{\min}, X_{\max}] \times [Y_{\min}, Y_{\max}]$. Therefore, we can express this constraint as follows

$$\begin{aligned} X_{\min} \le x_0(l,t) \le X_{\max}, \\ Y_{\min} \le y_0(l,t) \le Y_{\max}. \end{aligned} \tag{2}$$

The 3D coordinates of mobile user n at time slot t are denoted as $(x_n(l,t), y_n(l,t), 0)$. The distance between the UAV and mobile user n at time slot t is

$$\begin{aligned} &d_n(l,t) \\ &= \sqrt{(x_n(l,t)-x_0(l,t))^2 + (y_n(l,t)-y_0(l,t))^2 + H^2}. \end{aligned} \tag{3}$$

As demonstrated by many experiments, the likelihood of a line-of-sight (LoS) connection is very high when the altitude of the UAV is high. Therefore, we only consider the LoS connections and utilize the free-space path-loss model, as in many of the existing studies [33]. Specifically, the channel gain between the UAV and mobile user n at time slot t is

$$\begin{aligned} h_n(l,t) &= h_0 d_n(l,t)^{-\alpha} \\ &= \frac{h_0}{\left((x_n(l,t)-x_0(l,t))^2 + (y_n(l,t)-y_0(l,t))^2 + H^2\right)^{\alpha/2}}, \end{aligned} \tag{4}$$

where α is the path-loss exponent and h_0 is the channel gain at the reference distance.

B. Computing Model

At the beginning of time slot t, mobile user n generates one task $D_n(l,t) = (b_n(l,t), c_n(l,t))$, where $b_n(l,t)$ represents the task's size and $c_n(l,t)$ represents the required CPU cycles to complete the task. Note that $b_n(l,t)$ and $c_n(l,t)$ are random, depending on the underlying computing tasks. We use $I_n(l,t)$ to indicate whether or not to offload task $D_n(l,t)$ to the UAV. Specifically, $I_n(l,t) = 1$ indicates offloading the task, and $I_n(l,t) = 0$ indicates performing the task locally. We assume that each computation task can be completed within one time slot, as in [34], [35], [36]. We elaborate on the scenarios for task offloading and local computation, respectively, in the following.

1) *Task Offloading:* In this case, mobile users opting for offloading are required to transmit their tasks to the UAV. Upon receiving these offloaded tasks, the CPU cycles of the UAV are allocated appropriately to perform the tasks. Specifically, frequency-division multiple access is utilized to transmit the offloaded tasks, where the channel bandwidth W is evenly allocated. Therefore, the available channel bandwidth for mobile user n at time slot t is

$$w_n(l,t) = I_n(l,t)\frac{W}{\sum_{n=1}^{N} I_n(l,t)}. \tag{5}$$

The transmission rate from mobile user n to the UAV at time slot t is

$$R_n(l,t) = w_n(l,t)\log_2\left(1 + \frac{P_n h_n(l,t)}{\sigma^2 w_n(l,t)}\right), \tag{6}$$

where P_n is the constant transmit power of mobile user n and σ^2 is the noise power. At time slot t, the transmission delay required to offload the task from mobile user n is

$$D_n^{\mathrm{S}}(l,t) = \frac{b_n(l,t)}{R_n(l,t)}. \tag{7}$$

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:07:31 UTC from IEEE Xplore. Restrictions apply.

At time slot t, the execution duration of the offloaded task from mobile user n is

$$D_n^{\mathrm{C}}(l,t) = \frac{c_n(l,t)}{f_n^{\mathrm{U}}(l,t)}, \tag{8}$$

where $f_n^{\mathrm{U}}(l,t)$ denotes the allocated CPU frequency for mobile user n at time slot t. The delay caused by the UAV transmitting the generated results to the users is ignored, since the size of computing results is typically much smaller than that of computing tasks [37]. The total delay of task offloading from mobile user n at time slot t is

$$D_n^{\mathrm{U}}(l,t) = D_n^{\mathrm{S}}(l,t) + D_n^{\mathrm{C}}(l,t). \tag{9}$$

Due to the limited CPU frequency of the UAV, we have

$$\sum_{n=1}^{N} f_n^{\mathrm{U}}(l,t) \le F, \tag{10}$$

where F is the maximum CPU frequency of the UAV. As the number of computing cores in the UAV's processor is limited, at most M computing tasks can be performed simultaneously by the UAV [22]. Therefore, we have

$$\sum_{n=1}^{N} I_n(l,t) \le M. \tag{11}$$

The energy consumption of the UAV for performing the offloaded tasks during time slot t is

$$E^{\mathrm{C}}(l,t) = \sum_{n=1}^{N} I_n(l,t)\gamma f_n^{\mathrm{U}}(l,t)^3 D_n^{\mathrm{C}}(l,t), \tag{12}$$

where γ represents the effective switched capacitance [38]. The residual energy of UAV at time slot $t+1$ is

$$E(l,t+1) = E(l,t) - E^{\mathrm{C}}(l,t). \tag{13}$$

To ensure that all offloaded tasks can be completed, the UAV first calculates the energy required for computation. If the residual energy is not enough to meet the computation needs, i.e., $E(l,t+1) < 0$, then the UAV prevents the users from offloading their computation tasks. In such cases, the users have to perform the computation locally.

2) *Local Computation:* When either the channel quality is poor or the computing resources/energy of the UAV are insufficient, the users may choose to perform computation locally. The delay for user n to perform local computation at time slot t is

$$D_n^{\mathrm{O}}(l,t) = \frac{c_n(l,t)}{f_n}, \tag{14}$$

where f_n denotes the CPU frequency of mobile user n. The overall delay of the tasks at time slot t is [36], [39]

$$D(l,t) = \sum_{n=1}^{N} \left(1 - I_n(l,t)\right) D_n^{\mathrm{O}}(l,t) + I_n(l,t) D_n^{\mathrm{U}}(l,t). \tag{15}$$

C. Digital Twin Model and Problem Formulation

To obtain the real-time information of MEC that is essential for guiding the trajectory planning, computing resource allocation and offloading decision-making in the real world, a DT is maintained on the UAV. The DT serves as a virtual model of the physical world which can facilitate interactions with the physical world. Specifically, the DT of the system contains user DT and UAV DT. The user DT is constructed for each individual user, and stores its state. Specifically, the DT of user n at time slot t is denoted as

$$DT_n(l,t) = \{x_n(l,t), y_n(l,t), b_n(l,t), c_n(l,t), P_n, f_n\}, \tag{16}$$

where the parameters in $DT_n(l,t)$ are all related to user n. On the other hand, the UAV DT stores the UAV's state information, which at time slot t is denoted as

$$DT_0(l,t) = \{x_0(l,t), y_0(l,t), E(l,t), W, F, M, \gamma\}, \tag{17}$$

where the parameters in $DT_0(l,t)$ are all related to the UAV.

To establish the DT of the MEC network, the time-invariant parameters are first initialized, including the CPU frequencies (i.e., f_n) and transmit power (i.e., P_n) of mobile users, the UAV's CPU frequency (i.e., F), effective switched capacitance (i.e., γ), and the system bandwidth (i.e., W). Then, the mobile users in the physical world upload their time-varying parameters (i.e., the arrived tasks and positions) to the UAV. Next, the UAV integrates its position, remaining energy, and the state information reported by the mobile users to update their corresponding DT and subsequently optimize the decisions based on the state information from the DT.

We aim to minimize the system delay while considering the constrained computing resources and energy of the UAV. To this end, we jointly optimize the UAV's trajectory on a large timescale and the offloading decisions and computing resource allocation on a small timescale, which is formulated as follows

$$\min_{\substack{v(l),\alpha(l),\\ \boldsymbol{I}(l,t),\boldsymbol{f}(l,t)}} \quad \frac{1}{LT}\sum_{l=1}^{L}\sum_{t=1}^{T} D(l,t) \tag{18}$$

$$\text{s.t.} \quad 0 \le v(l) \le V_{\max}, \forall\, l \in \mathcal{L}, \tag{18a}$$

$$0 \le \alpha(l) < 2\pi, \forall\, l \in \mathcal{L}, \tag{18b}$$

$$X_{\min} \le x(l,t) \le X_{\max}, \forall\, l \in \mathcal{L}, \forall\, t \in \mathcal{T}, \tag{18c}$$

$$Y_{\min} \le y(l,t) \le Y_{\max}, \forall\, l \in \mathcal{L}, \forall\, t \in \mathcal{T}, \tag{18d}$$

$$I_n(l,t) \in \{0,1\}, \forall\, l \in \mathcal{L}, \forall\, t \in \mathcal{T}, \tag{18e}$$

$$\sum_{n=1}^{N} I_n(l,t) \le M, \forall\, l \in \mathcal{L}, \forall\, t \in \mathcal{T}, \tag{18f}$$

$$\sum_{n=1}^{N} f_n^{\mathrm{U}}(l,t) \le F, \forall\, l \in \mathcal{L}, \forall\, t \in \mathcal{T}, \tag{18g}$$

$$E(l,t) \ge 0, \forall\, l \in \mathcal{L}, \forall\, t \in \mathcal{T}, \tag{18h}$$

$$E(1,1) = E, \tag{18i}$$

where $\boldsymbol{I}(l,t) = \{I_n(l,t)\}_{N\times 1}$ represents the offloading decisions at time slot t of frame l, $\boldsymbol{f}(l,t) = \{f_n^{\mathrm{U}}(l,t)\}_{N\times 1}$ denotes the allocated CPU frequencies to mobile users at time slot t of frame l, $V_{\max}$ represents the maximum flight speed of the UAV, $\mathcal{L} = \{1,2,\ldots,L\}$ and $\mathcal{T} = \{1,2,\ldots,T\}$ are defined as

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:07:31 UTC from IEEE Xplore. Restrictions apply.

the sets of indices, and $E(1,1)$ denotes the initial energy of the UAV. Constraints (18a), (18b), (18c), and (18d) correspond to the restrictions on the UAV's trajectory, while constraints (18f) and (18g) pertain to the limitations of the UAV's computing resources. Additionally, constraints (18h) and (18i) define the limitations on the energy of the UAV.

Problem (18) is highly challenging due to the following reasons. Firstly, the users' locations, sizes of arrived tasks, and channel conditions are time-varying, and thus their complete knowledge in all time slots is impractical to be obtained beforehand, making it difficult for traditional optimization-based methods to solve the problem. Secondly, as the optimization problem involves two different timescales, the conventional DRL algorithms typically involve only one timescale and thus cannot be directly applied to solve the problem. Lastly, the variables related to the computing resource constraints are coupled, including discrete variable $\boldsymbol{I}(l,t)$ and continuous variable $\boldsymbol{f}(l,t)$, making constraints (18f) and (18g) non-convex. To effectively tackle the problem, we model it as a two-timescale MDP and propose a customized DRL algorithm.

III. MDP Modeling

In this section, we model the problem as an MDP. As problem (18) involves two timescales, the design of state spaces, action spaces, and reward functions for the large and small timescales are provided in the following, respectively.

A. Large Timescale Trajectory Planning

On the large timescale, the UAV obtains system state information from DT at the beginning of each frame and determines the speed and direction of flight. The corresponding elements in MDP are defined as follows.

- *State:* The state at frame l is defined as

$$\boldsymbol{S}_{\mathcal{L}}(l) = \{\boldsymbol{x}(l), \boldsymbol{y}(l), x_0(l,1), y_0(l,1), E(l,1)\}, \quad (19)$$

  where $\boldsymbol{x}(l) = \{x_n(l,1)\}_{N\times 1}$ and $\boldsymbol{y}(l) = \{y_n(l,1)\}_{N\times 1}$. As the diverse size of state variables may lead to difficulty in algorithm convergence [26], we normalize the state as

$$\overline{\boldsymbol{S}}_{\mathcal{L}}(l) = \left\{ \frac{\boldsymbol{x}(l)}{X_{\max}}, \frac{\boldsymbol{y}(l)}{Y_{\max}}, \frac{x_0(l,1)}{X_{\max}}, \frac{y_0(l,1)}{Y_{\max}}, \frac{E(l,1)}{E} \right\}. \quad (20)$$

- *Action:* The action at frame l is defined as

$$\boldsymbol{a}_{\mathcal{L}}(l) = \{v(l), \alpha(l)\}. \quad (21)$$

  The ranges of flight speed and direction for the UAV are changed to $[-V_{\max}, V_{\max}]$ and $[-\pi, \pi]$, which is experimentally proved to perform well [40].
- *Reward:* Since we aim to minimize the system delay, the reward for frame l is the negative sum of the delays of all time slots in frame l and denoted as

$$r_{\mathcal{L}}(l) = -\sum_{t=1}^{T} D(l,t). \quad (22)$$

B. Small Timescale Offloading Decision and Computing Resources Allocation

On the small timescale, the UAV obtains the system state information from DT at the beginning of each time slot and optimizes $\boldsymbol{I}(l,t)$ and $\boldsymbol{f}(l,t)$. The corresponding elements in MDP are defined as follows.

- *State:* The state at time slot t is defined as

$$\boldsymbol{S}_{\mathcal{S}}(l,t) = \{\boldsymbol{x}(l,t), \boldsymbol{y}(l,t), \boldsymbol{b}(l,t), \boldsymbol{c}(l,t), x_0(l,t), y_0(l,t), E(l,t)\}, \quad (23)$$

  where $\boldsymbol{x}(l,t) = \{x_n(l,t)\}_{N\times 1}$, $\boldsymbol{y}(l,t) = \{y_n(l,t)\}_{N\times 1}$, $\boldsymbol{b}(l,t) = \{b_n(l,t)\}_{N\times 1}$, and $\boldsymbol{c}(l,t) = \{c_n(l,t)\}_{N\times 1}$. Similar to (20), we normalize the state as in (24) shown at the bottom of the page, where $b_{\max} = \max\{b(l,t)|\forall\, l \in \mathcal{L}, \forall\, t \in \mathcal{T}\}$ and $c_{\max} = \max\{c(l,t)|\forall\, l \in \mathcal{L}, \forall\, t \in \mathcal{T}\}$.
- Action: The action at time slot t includes the offloading decisions and the CPU frequency allocation, and is defined as

$$\boldsymbol{a}_{\mathcal{S}}(l,t) = \{\boldsymbol{I}(l,t), \boldsymbol{f}(l,t)\}. \quad (25)$$

  Note that action $\boldsymbol{a}_{\mathcal{S}}(l,t)$ contains both discrete variable $\boldsymbol{I}(l,t)$ and continuous variable $\boldsymbol{f}(l,t)$, which cannot be directly solved by existing DRL algorithms. To deal with this issue, we use $f_n^{\mathrm{U}}(l,t)$ to implicitly represent $I_n(l,t)$. Specifically, $f_n^{\mathrm{U}}(l,t) = 0$ indicates that user n performs computation locally (i.e., $I_n(l,t) = 0$), and $f_n^{\mathrm{U}}(l,t) > 0$ indicates offloading the task (i.e., $I_n(l,t) = 1$). Then, action $\boldsymbol{a}_{\mathcal{S}}(l,t)$ can be simplified as

$$\boldsymbol{a}_{\mathcal{S}}(l,t) = \boldsymbol{f}(l,t). \quad (26)$$

- Reward: Similar to (22), the reward for time slot t is

$$r_{\mathcal{S}}(l,t) = -D(l,t). \quad (27)$$

C. Two-Timescale MDP

As shown in Fig. 2, we extend the standard MDP into a two-timescale MDP, where the policies for the large and small timescales are denoted as $\boldsymbol{\pi}_{\mathcal{L}}$ and $\boldsymbol{\pi}_{\mathcal{S}}$, respectively. Specifically, at the beginning of frame 1, the UAV obtains state $\overline{\boldsymbol{S}}_{\mathcal{L}}(1)$ through the DT and then takes action $\boldsymbol{a}_{\mathcal{L}}(1)$. Similarly, at the beginning of time slot $(1,1)$, the UAV obtains state $\overline{\boldsymbol{S}}_{\mathcal{S}}(1,1)$ through the DT and then takes action $\boldsymbol{a}_{\mathcal{S}}(1,1)$. Then, the UAV receives a reward $r_{\mathcal{S}}(1,1)$, and the state at the small timescale transits from $\overline{\boldsymbol{S}}_{\mathcal{S}}(1,1)$ to $\overline{\boldsymbol{S}}_{\mathcal{S}}(1,2)$. After executing actions for the small timescale over of T time slots, the UAV receives a reward $r_{\mathcal{L}}(1)$

$$\overline{\boldsymbol{S}}_{\mathcal{S}}(l,t) = \left\{ \frac{\boldsymbol{x}(l,t)}{X_{\max}}, \frac{\boldsymbol{y}(l,t)}{Y_{\max}}, \frac{\boldsymbol{b}(l,t)}{b_{\max}}, \frac{\boldsymbol{c}(l,t)}{c_{\max}}, \frac{x_0(l,t)}{X_{\max}}, \frac{y_0(l,t)}{Y_{\max}}, \frac{E(l,t)}{E} \right\} \quad (24)$$

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:07:31 UTC from IEEE Xplore. Restrictions apply.

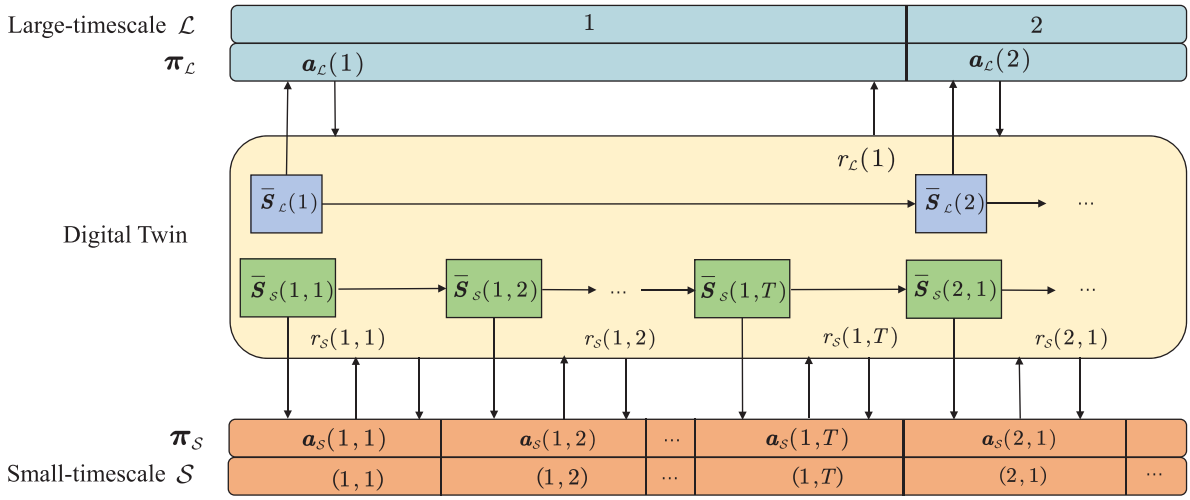


Fig. 2. Illustration of the proposed two-timescale MDP for DT-assisted MEC.

for taking action $\boldsymbol{a}_{\mathcal{L}}(1)$, which is the cumulative reward for this frame. Meanwhile, the state at the large timescale transits from $\overline{\boldsymbol{S}}_{\mathcal{L}}(1)$ to $\overline{\boldsymbol{S}}_{\mathcal{L}}(2)$. Then, the MDP iterates until convergence.

According to the structure of the proposed two-timescale MDP, the objective function of problem (18) on the large timescale at frame l can be expressed as

$$\max_{\boldsymbol{\pi}_{\mathcal{L}}} \mathbb{E}\left[\sum_{i=0}^{L-l} \eta^i r_{\mathcal{L}}(l+i)\right], \tag{28}$$

where η is a discount factor. The objective function of problem (18) at time slot t of frame l on the small timescale can be expressed as

$$\max_{\boldsymbol{\pi}_{\mathcal{S}}} \mathbb{E}\left[\sum_{i=0}^{T-t}\sum_{j=0}^{L-l} \eta^{jT+i} r_{\mathcal{S}}(l+j,t+i)\right]. \tag{29}$$

IV. Proposed Algorithm

As the distribution of arrived tasks and the direction and speed of users' movement are unknown, it is necessary to propose a model-free algorithm that does not need prior knowledge of the state transition probabilities. Since the state variables (i.e., size of arrived tasks, energy of the UAV) and allocation of CPU frequency are continuously valued, tabular-based algorithms such as Q-learning [41] are unsuitable to be applied. Meanwhile, the deep deterministic policy gradient (DDPG) algorithm suffers from high variance, which may lead to an unstable training process [42]. In comparison, the PPO algorithm is more stable than DDPG by limiting the difference between the new and old policies during iterations and performs well in many MDP problems, especially for the trajectory optimization of UAVs [40]. Therefore, we resort to adopt the PPO algorithm.

A. Customized PPO Algorithm

The PPO algorithm is based on the actor-critic framework, with a policy network to generate actions, and a critic network to evaluate the value of actions. The proposed algorithm is summarized in Algorithm 1, which is customized to deal with the two-timescale optimization problem, with a policy network and a critic network for each timescale, as shown in Fig. 3. Additionally, the actions generated by the PPO algorithm are projected by the safe projection method to satisfy the computing resource constraints (Line 17), which will be introduced in Section IV-B. On the large timescale, the UAV trajectory is planned by the policy network for the large timescale, taking into account the speed and range constraints (Lines 5-13). On the small timescale, the offloading decisions and the computing resource allocation are optimized by the policy network for the small timescale, while ensuring the satisfaction of the energy constraints (Lines 14-25). After completing each trajectory, policies $\boldsymbol{\pi}_{\mathcal{L}}$ and $\boldsymbol{\pi}_{\mathcal{S}}$, as well as the critic networks are updated (Lines 30-33).

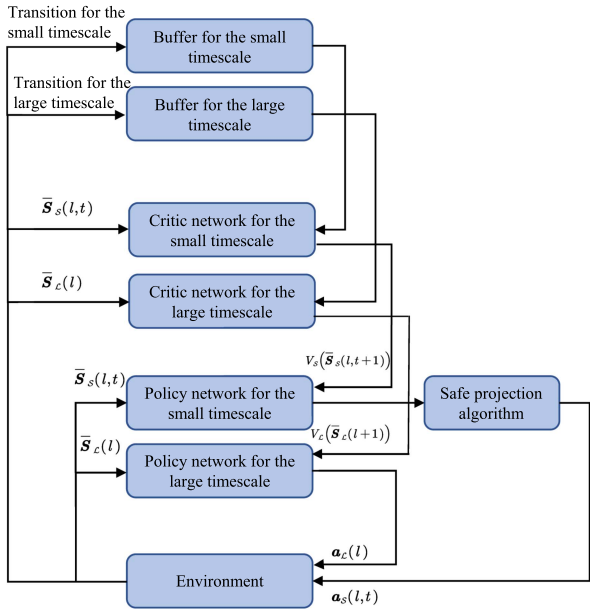


Fig. 3. Illustration of the proposed algorithm.

In the following, the loss functions of the policy networks and critic networks for the large and small timescales are respectively designed. The objective function of the problem on the large timescale can be expressed as the state-action value function

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:07:31 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Proposed Customized Two-timescale PPO Algorithm.

Input: States $\overline{\boldsymbol{S}}_{\mathcal{L}}(l)$, $\overline{\boldsymbol{S}}_{\mathcal{S}}(l,t)$.
Output: Actions $\boldsymbol{a}_{\mathcal{L}}(l)$, $\boldsymbol{a}_{\mathcal{S}}(l,t)$.
Initialize policy networks and critic networks;
for episode=1 : Episode **do**
  Initialize environment;
  for $l \in \{1,2,\ldots,L\}$ **do**
    Observe state $\overline{\boldsymbol{S}}_{\mathcal{L}}(l)$);
    Obtain $\boldsymbol{O}_{\mathcal{L}}(l) = \{O_{\mathcal{L}}^1, O_{\mathcal{L}}^2\}$ based on $\boldsymbol{\pi}_{\mathcal{L}}$;
    Obtain $\boldsymbol{a}_{\mathcal{L}}(l)$ by using $v(l) = clip\{1,-1,O_{\mathcal{L}}^1\} \times V_{\max}$ and $\alpha(l) = clip\{1,-1,O_{\mathcal{L}}^2\} \times \pi$;
    if $x(l+1,1)$ violates constraint (18d) or $y(l+1,1)$ violates constraint (18e) **then**
      $\boldsymbol{a}_{\mathcal{L}}(l) = \mathbf{0}$;
    end if
    Execute action $\boldsymbol{a}_{\mathcal{L}}(l)$;
    for $t \in \{1,2,\ldots,T\}$ **do**
      Observe state $\overline{\boldsymbol{S}}_{\mathcal{S}}(l,t)$);
      Obtain $\boldsymbol{O}_{\mathcal{S}}(l,t)$ based on $\boldsymbol{\pi}_{\mathcal{S}}$;
      Obtain $\boldsymbol{a}_{\mathcal{S}}(l,t)$ by Algorithm 2;
      Compute energy $E(l,t+1)$ via (13);
      Compute $\Delta(l,t) = \sum_{n=1}^{N} D_n^O(l,t)$;
      if $E(l,t+1) < 0$ or $\Delta(l,t) < D(l,t)$ **then**
        $\boldsymbol{a}_{\mathcal{S}}(l,t) = \mathbf{0}$;
      end if
      Execute action $\boldsymbol{a}_{\mathcal{S}}(l,t)$;
      Receive reward $r_{\mathcal{S}}(l,t)$ and observe next state $\overline{\boldsymbol{S}}_{\mathcal{S}}(l,t+1)$;
      Store transition $(\overline{\boldsymbol{S}}_{\mathcal{S}}(l,t), \boldsymbol{a}_{\mathcal{S}}(l,t), r_{\mathcal{S}}(l,t), \overline{\boldsymbol{S}}_{\mathcal{S}}(l,t+1))$ into buffer $B_{\mathcal{S}}$;
    end for
    Receive reward $r_{\mathcal{L}}(l)$ and observe next state $\overline{\boldsymbol{S}}_{\mathcal{L}}(l+1)$;
    Store transition $(\overline{\boldsymbol{S}}_{\mathcal{L}}(l), \boldsymbol{a}_{\mathcal{L}}(l), r_{\mathcal{L}}(l), \overline{\boldsymbol{S}}_{\mathcal{L}}(l+1))$ into buffer $B_{\mathcal{L}}$;
  end for
  Update policy $\boldsymbol{\pi}_{\mathcal{L}}$ by maximizing $Loss_{\mathcal{L}}^{\text{policy}}$ via policy gradient ascent;
  Update policy $\boldsymbol{\pi}_{\mathcal{S}}$ by maximizing $Loss_{\mathcal{S}}^{\text{policy}}$ via policy gradient ascent;
  Minimizing $Loss_{\mathcal{L}}^{\text{critic}}$ via gradient descent;
  Minimizing $Loss_{\mathcal{S}}^{\text{critic}}$ via gradient descent;
  Reset $B_{\mathcal{L}}$ and $B_{\mathcal{S}}$.
end for

$Q_{\mathcal{L}}(\overline{\boldsymbol{S}}_{\mathcal{L}}(l), \boldsymbol{a}_{\mathcal{L}}(l))$, given by

$$Q_{\mathcal{L}}\left(\overline{\boldsymbol{S}}_{\mathcal{L}}(l), \boldsymbol{a}_{\mathcal{L}}(l)\right) = \mathbb{E}\left[\sum_{i=0}^{L-l} \eta^i r_{\mathcal{L}}(l+i) | \overline{\boldsymbol{S}}_{\mathcal{L}}(l), \boldsymbol{a}_{\mathcal{L}}(l)\right]. \tag{30}$$

To minimize the delay, the state-action value function $Q_{\mathcal{L}}(\overline{\boldsymbol{S}}_{\mathcal{L}}(l), \boldsymbol{a}_{\mathcal{L}}(l))$ should be maximized by executing the optimal flight action $\boldsymbol{a}_{\mathcal{L}}^*(l)$. As directly using the state-action value function as the loss function of the policy network leads to high variance, PPO adopts the advantage function instead, which can directly reflect the contribution of each action by measuring the advantage or disadvantage of an action relative to the average level. To balance bias and variance, generalized advantage estimator $\hat{A}_{\mathcal{L}}(\overline{\boldsymbol{S}}_{\mathcal{L}}(l), \boldsymbol{a}_{\mathcal{L}}(l))$, given in (31) shown at the bottom of the page, is adopted to estimate the advantage function [43], where λ is a hyper-parameter to balance the bias and variance, and $V_{\mathcal{L}}(\overline{\boldsymbol{S}}_{\mathcal{L}}(l))$ is the state value function. Note that $V_{\mathcal{L}}(\overline{\boldsymbol{S}}_{\mathcal{L}}(l))$ measures the expected cumulative reward that can be obtained from state $\overline{\boldsymbol{S}}_{\mathcal{L}}(l)$, given by

$$V_{\mathcal{L}}\left(\overline{\boldsymbol{S}}_{\mathcal{L}}(l)\right) = \mathbb{E}\left[\sum_{i=0}^{L-l} \eta^i r_{\mathcal{L}}(l+i) | \overline{\boldsymbol{S}}_{\mathcal{L}}(l)\right]. \tag{32}$$

Note that $V_{\mathcal{L}}(\overline{\boldsymbol{S}}_{\mathcal{L}}(l+1))$ and $V_{\mathcal{L}}(\overline{\boldsymbol{S}}_{\mathcal{L}}(l))$ are estimated by the critic network for the large timescale.

The PPO algorithm updates policies iteratively through policy gradient ascent. To improve the stability of the algorithm during iteration, PPO restricts the differences between the new policy of the current episode and the old policy of the previous episode to avoid excessive policy updates. We denote the flight policy obtained from the previous episode for the large timescale as $\boldsymbol{\pi}_{\boldsymbol{\theta}'_{\mathcal{L}}}$, where $\boldsymbol{\theta}'_{\mathcal{L}}$ is the parameter of the policy network for the large timescale in the previous episode. To measure the difference between the new flight policy $\boldsymbol{\pi}_{\boldsymbol{\theta}_{\mathcal{L}}}$ and the old flight policy $\boldsymbol{\pi}_{\boldsymbol{\theta}'_{\mathcal{L}}}$, we define the probability ratio as

$$\xi(\boldsymbol{\theta}_{\mathcal{L}}) = \frac{\boldsymbol{\pi}_{\boldsymbol{\theta}_{\mathcal{L}}}\left(\boldsymbol{a}_{\mathcal{L}}(l) | \overline{\boldsymbol{S}}_{\mathcal{L}}(l)\right)}{\boldsymbol{\pi}_{\boldsymbol{\theta}'_{\mathcal{L}}}\left(\boldsymbol{a}_{\mathcal{L}}(l) | \overline{\boldsymbol{S}}_{\mathcal{L}}(l)\right)}, \tag{33}$$

where $\boldsymbol{\theta}_{\mathcal{L}}$ is the parameter of the policy network for the large timescale in the current episode. PPO restricts excessive policy updates by clipping the loss function. Specifically, the loss function of the policy network for the large timescale is given in (34) shown at the bottom of the next page, where ϵ is a hyper-parameter used to control the difference between the policies, and the clip function restricts the value of $\xi(\boldsymbol{\theta}_{\mathcal{L}})$ to be within the range of $1-\epsilon$ and $1+\epsilon$. Specifically, we have

$$\text{clip}\left(\xi(\boldsymbol{\theta}_{\mathcal{L}}), 1-\epsilon, 1+\epsilon\right)$$

$$\begin{aligned}\hat{A}_{\mathcal{L}}\left(\overline{\boldsymbol{S}}_{\mathcal{L}}(l), \boldsymbol{a}_{\mathcal{L}}(l)\right) &= \sum_{i=0}^{L-l} (\eta\lambda)^i \left(Q_{\mathcal{L}}\left(\overline{\boldsymbol{S}}_{\mathcal{L}}(l+i), \boldsymbol{a}_{\mathcal{L}}(l+i)\right) - V_{\mathcal{L}}\left(\overline{\boldsymbol{S}}_{\mathcal{L}}(l+i)\right)\right) \\ &= \sum_{i=0}^{L-l} (\eta\lambda)^i \left(r_{\mathcal{L}}(l+i) + \eta V_{\mathcal{L}}\left(\overline{\boldsymbol{S}}_{\mathcal{L}}(l+i+1)\right) - V_{\mathcal{L}}\left(\overline{\boldsymbol{S}}_{\mathcal{L}}(l+i)\right)\right)\end{aligned} \tag{31}$$

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:07:31 UTC from IEEE Xplore. Restrictions apply.

$$= \begin{cases} 1-\epsilon, & \text{if } \xi(\boldsymbol{\theta}_{\mathcal{L}}) < 1-\epsilon, \\ 1+\epsilon, & \text{if } \xi(\boldsymbol{\theta}_{\mathcal{L}}) > 1+\epsilon, \\ \xi(\boldsymbol{\theta}_{\mathcal{L}}), & \text{if } 1-\epsilon \le \xi(\boldsymbol{\theta}_{\mathcal{L}}) \le 1+\epsilon. \end{cases} \tag{35}$$

Note that the policy network for the large timescale generates the means and variances of flight action $\boldsymbol{a}_{\mathcal{L}}(l)$. To ensure that flight action $\boldsymbol{a}_{\mathcal{L}}(l)$ satisfies the constraints of flight speed and direction, we clip the action generated by the policy network for the large timescale, as in line 9 of Algorithm 1, where $\boldsymbol{O}_{\mathcal{L}}(l)$ is the action sampled from a normal distribution based on the generated means and variances of the policy network.

Temporal difference algorithm is adopted to estimate the state value functions. Therefore, the loss function of the critic network for the large timescale is given by

$$Loss_{\mathcal{L}}^{\text{critic}} = \mathbb{E}\left[\left(V_{\mathcal{L}}\left(\overline{\boldsymbol{S}}_{\mathcal{L}}(l)\right) - r_{\mathcal{L}}(l) - \eta V_{\mathcal{L}}\left(\overline{\boldsymbol{S}}_{\mathcal{L}}(l+1)\right)\right)^2\right]. \tag{36}$$

Similar to (34), the loss function of the policy network for the small timescale is given in (37) shown at the bottom of the page, where $\hat{A}_{\mathcal{S}}(\overline{\boldsymbol{S}}_{\mathcal{S}}(l,t), \boldsymbol{a}_{\mathcal{S}}(l,t))$ is the generalized advantage estimator for the small timescale, and $\boldsymbol{\theta}_{\mathcal{S}}$ is the parameter of the policy network for the small timescale in the current episode. To make the training process more stable, we restrict the UAV from taking some suboptimal actions on the small timescale. Specifically, before allocating computing resources to the users, the UAV calculates the delay caused by the action $\boldsymbol{a}_{\mathcal{S}}(l,t)$. If the delay is greater than the local computing delay, all tasks at time slot t are computed locally (Lines 19-21 of Algorithm 1).

Similarly to (36), the loss function of the critic network for the small timescale is designed based on the temporal difference algorithm [44] as follows

$$Loss_{\mathcal{S}}^{\text{critic}} = \mathbb{E}\Big[\Big(V_{\mathcal{S}}\left(\overline{\boldsymbol{S}}_{\mathcal{S}}(l,t)\right) - r_{\mathcal{S}}(l,t) - \eta V_{\mathcal{S}}\left(\overline{\boldsymbol{S}}_{\mathcal{S}}(l,t+1)\right)\Big)^2\Big], \tag{38}$$

where $V(\overline{\boldsymbol{S}}_{\mathcal{S}}(l,t))$ is the state value function for the small timescale. The proposed algorithm involves two timescales, where the optimization of the large timescale relies on the optimization of the small timescale. The convergence analysis of the proposed algorithm is similar to that in [45].

B. Safe Projection Algorithm

In this subsection, we propose a projection-based method to deal with the computing resource constraints (18f) and (18g). Most existing works tackle constraints by punishing actions that lead to constraint violations [22], [23], [24], [25]. However, this method cannot guarantee that the constraints are always satisfied during the training process and requires selecting an appropriate penalty value. To overcome these shortcomings, we design a safe projection method to handle the computing resource constraints. As the policy network for the small timescale generates the means and variances of action $\boldsymbol{a}_{\mathcal{S}}(l,t)$, we first sample the action from the normal distribution based on the means and variances generated by the policy network and then map the action to a feasible action.

Algorithm 2: Safe Projection Algorithm.

1: **Input:** Action sampled from the policy network $\boldsymbol{O}_{\mathcal{S}}(l,t)$.
2: **Output:** Action on the small time scale of the UAV, i.e, $\boldsymbol{a}_{\mathcal{S}}(l,t)$.
3: $f^{\text{Sum}}(l,t) = \min\{|\frac{\sum_{j=1}^{J} f_j^{os}(l,t)}{J}|, 1\} \times F$;
4: $M^{\text{Sum}}(l,t) = \min\{round(|\frac{\sum_{j=1}^{J} M_j^{os}(l,t)}{J}|M), M\}$;
5: Find the $M^{\text{Sum}}(l,t)$-th largest value $f^*(l,t)$ of $|\boldsymbol{f}^o(l,t)|$;
6: Set the values in $|\boldsymbol{f}^o(l,t)|$ to zero for all elements except the largest $M^{\text{Sum}}(l,t)$;
7: $\boldsymbol{f}(l,t) = \frac{|\boldsymbol{f}^o(l,t)|}{\sum_{n=1}^{N}|f_n^o(l,t)|} f^{\text{Sum}}(l,t)$;
8: $\boldsymbol{a}_{\mathcal{S}}(l,t) = \boldsymbol{f}(l,t)$.

Note that the computing resource constraints cannot be directly taken as equals since allocating more computing resources can reduce delay but may consume more energy. Therefore, it is not feasible to satisfy these constraints by directly normalizing the actions. To convert these two inequality constraints into equality constraints, the agent needs to learn two additional variables: the total number of users performing computation offloading to the UAV at time slot t, denoted as

$$M^{\text{Sum}}(l,t) = \sum_{n=1}^{N} I_n(l,t), \tag{39}$$

and the sum of CPU frequency allocated by the UAV to all users at time slot t, denoted as

$$f^{\text{Sum}}(l,t) = \sum_{n=1}^{N} f_n^{\text{U}}(l,t). \tag{40}$$

To ensure that constraints (18f) and (18g) are satisfied, $M^{\text{Sum}}(l,t)$ and $f^{\text{Sum}}(l,t)$ need to satisfy

$$M^{\text{Sum}}(l,t) \in \{0, 1, \ldots, M\}, \tag{41}$$

and

$$0 \le f^{\text{Sum}}(l,t) \le F. \tag{42}$$

The proposed safe projection method consists of three steps. The first step is to determine the total number of users that offload the task (i.e., $M^{\text{Sum}}(l,t)$) and the sum of CPU frequency allocated to all users (i.e., $f^{\text{Sum}}(l,t)$). The second step is to

$$Loss_{\mathcal{L}}^{\text{policy}}(\boldsymbol{\theta}_{\mathcal{L}}) = \mathbb{E}\left[\min\left(\xi(\boldsymbol{\theta}_{\mathcal{L}})\hat{A}_{\mathcal{L}}\left(\overline{\boldsymbol{S}}_{\mathcal{L}}(l), \boldsymbol{a}_{\mathcal{L}}(l)\right), \text{clip}\left(\xi(\boldsymbol{\theta}_{\mathcal{L}}), 1-\epsilon, 1+\epsilon\right)\hat{A}_{\mathcal{L}}\left(\overline{\boldsymbol{S}}_{\mathcal{L}}(l), \boldsymbol{a}_{\mathcal{L}}(l)\right)\right)\right] \tag{34}$$

$$Loss_{\mathcal{S}}^{\text{policy}}(\boldsymbol{\theta}_{\mathcal{S}}) = \mathbb{E}\left[\min\left(\xi(\boldsymbol{\theta}_{\mathcal{S}})\hat{A}_{\mathcal{S}}\left(\overline{\boldsymbol{S}}_{\mathcal{S}}(l,t), \boldsymbol{a}_{\mathcal{S}}(l,t)\right), \text{clip}\left(\xi(\boldsymbol{\theta}_{\mathcal{S}}), 1-\epsilon, 1+\epsilon\right)\hat{A}_{\mathcal{S}}\left(\overline{\boldsymbol{S}}_{\mathcal{S}}(l,t), \boldsymbol{a}_{\mathcal{S}}(l,t)\right)\right)\right] \tag{37}$$

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:07:31 UTC from IEEE Xplore. Restrictions apply.

determine which user's task to offload to the UAV. The third step is to allocate computing resources to the users. To design the policy network for action $\boldsymbol{a}_{\mathcal{S}}(l,t)$, we need to determine the dimension of the output layer of the policy network. Usually, the dimension of the output layer is twice that of the action (i.e., each variable in the action is assigned two neurons, one that generates the mean of the variable and the other that generates the variance of the variable). However, as $M^{\text{Sum}}(l,t)$ and $f^{\text{Sum}}(l,t)$ affect the CPU frequency allocation in the next step, these two variables hold significant importance. Therefore, we assign $2J$ neurons to each of these two variables [25], which may improve the performance of the algorithm. Note that J is a hyper-parameter that satisfies $J > 1$. In addition, variable $f_n^{\text{U}}(l,t)$ is assigned two neurons. Therefore, the dimension of the output layer of the policy network is $4J + 2N$. The action sampled from the normal distribution based on the output of the policy network for the small timescale is denoted as

$$\boldsymbol{O}_{\mathcal{S}}(l,t) = \{\boldsymbol{M}^{os}(l,t), \boldsymbol{f}^{os}(l,t), \boldsymbol{f}^{o}(l,t)\}, \tag{43}$$

where $\boldsymbol{M}^{os}(l,t) = \{M_j^{os}(l,t)\}_{J\times 1}$ is used to obtain $M^{\text{Sum}}(l,t)$, $\boldsymbol{f}^{os}(l,t) = \{f_j^{os}(l,t)\}_{J\times 1}$ is used to obtain $f^{\text{Sum}}(l,t)$, and $\boldsymbol{f}^{o}(l,t) = \{f_n^{o}(l,t)\}_{N\times 1}$ is used to obtain $\boldsymbol{f}(l,t)$.

Then, we map action $\boldsymbol{O}_{\mathcal{S}}(l,t)$ to a feasible action using the proposed safe projection method, which is summarized in Algorithm 2. Firstly, the total CPU frequency allocated to all users (i.e., $f^{\text{Sum}}(l,t)$) is obtained in Line 3. As the activation function of the output layer for the means is set as the Tanh function, the range of $f_j^{os}(l,t)$ is $(-1,1)$ when the variances approach to zero, and it can be concluded that

$$0 < \mathbb{E}\left(\left|\frac{\sum_{j=1}^{J} f_j^{os}(l,t)}{J}\right|\right) < 1. \tag{44}$$

As $|\frac{\sum_{j=1}^{J} f_j^{os}(l,t)}{J}| > 1$ may also happen when the variances are relatively large, especially in the initial stage of training, we take the minimum between $|\frac{\sum_{j=1}^{J} f_j^{os}(l,t)}{J}|$ and 1in Line 3 to avoid this case. Therefore, we ensure $0 < f^{\text{Sum}}(l,t) < F$. Next, the total number of users that offload computation to the UAV (i.e., $M^{\text{Sum}}(l,t)$) is obtained in Line 4. Similarly, it can be deduced that the range of $M^{\text{Sum}}(l,t)$ is $\{0,1,\ldots,M\}$. Then, we decide which users' tasks to offload to the UAV. As the allocated CPU frequencies are non-negative, we first take the absolute value of $\boldsymbol{f}^{o}(l,t)$. Similar to [22], we consider that a larger value of $|f_n^{o}(l,t)|$ indicates a greater likelihood of user n offloading computation to the UAV. Therefore, computing resources are allocated only to the top $M^{\text{Sum}}(l,t)$ users with the highest $|f_n^{o}(l,t)|$ values, as shown in Line 5 and Line 6. Note that the Quick Select algorithm is used to find the $M^{\text{Sum}}(l,t)$-th largest value in Line 5 because it has the lowest complexity, which is $\mathcal{O}(N)$ [46]. Finally, the CPU frequency allocated to each user is obtained by normalizing $|\boldsymbol{f}^{o}(l,t)|$, as shown in Line 7. If $|f_n^{o}(l,t)|$ is set to zero, user n performs computation locally. In summary, constraint (18f) is satisfied by executing Lines 4-6, while constraint (18g) is satisfied by executing Line 3 and Line 7.

TABLE I
SYSTEM PARAMETERS

Description	Value
Number of users	$N = 10$
Number of CPU cores on the UAV	$M = 9$
Maximum flight speed of the UAV	$V_{\max} = 500$ m/frame
Energy of the UAV	$E(1,1) = 20,000$ J
Bandwidth	$W = 1$ MHz
Noise power	$\sigma^2 = 0.001$ W
Maximum CPU frequency of the UAV	$F = 5$ GHz
Number of frames	$L = 10$
Number of time slots per frame	$T = 3$
Effective switched capacitance	$\gamma = 1$
Discount factor	$\eta = 0.99$
Discount hyper-parameter	$\lambda = 0.99$
Policy update coefficient	$\epsilon = 0.2$
CPU frequency of user	$f_n \in (10,20)$ MHz
Task size	$b_n(l,t) \in (100,200)$ Kbits
CPU cycles required by the task	$c_n(l,t) \in (20,30)$ M
Height of the UAV	$H = 20$ m
Path-loss exponent	$\alpha = 2$

C. Complexity Analysis

In each time slot, the UAV makes the offloading decisions and allocates computing resources. This action consists of two steps. The first step is to obtain $\boldsymbol{O}_{\mathcal{S}}(l,t)$ based on the current state $\overline{\boldsymbol{S}}_{\mathcal{S}}(l,t)$ through the policy network for the small timescale, with a complexity of $\mathcal{O}(\sum_{p=1}^{P} n_{p-1}n_p)$, where n_p is the number of neurons at layer p of the policy network for the small timescale. The second step is to map $\boldsymbol{O}_{\mathcal{S}}(l,t)$ to action $\boldsymbol{a}_{\mathcal{S}}(l,t)$ using Algorithm 2. As the complexity of Algorithm 2 mainly lies in executing the Quick Select algorithm, the complexity of Algorithm 2 is $\mathcal{O}(N)$. Therefore, the computational complexity for each time slot is $\mathcal{O}(N + \sum_{p=1}^{P} n_{p-1}n_p)$. On the large timescale, the UAV decides its flight speed and direction per frame through the policy network for the large timescale, with a complexity of $\mathcal{O}(\sum_{q=1}^{Q} n_{q-1}n_q)$, where n_q is the number of neurons at layer q of the policy network for the large timescale. Therefore, the overall computational complexity of the proposed algorithm is $\mathcal{O}(LT(N + \sum_{p=1}^{P} n_{p-1}n_p) + L\sum_{q=1}^{Q} n_{q-1}n_q)$.

V. SIMULATION RESULTS

This section presents simulation resuls to validate the convergence and superiority of the proposed algorithm and elucidate the impact of various parameters on the average delay.

A. Simulation Setup

The UAV-assisted MEC network consists of $N = 10$ mobile users and a UAV. The UAV and users are moving within a square area of $[0,2000] \times [0,2000]$ meters, with the UAV's initial coordinates being $(1000,1000,15)$ meters. The movement of mobile users is modeled by the Gauss-Markov random model [47]. Other simulation parameters are listed in Table I.

The simulation is executed on an NVIDIA GTX 3090 GPU, and the PyTorch version is 2.0.0. Both the policy networks and critic networks have three hidden layers, with each hidden layer containing 256 neurons. Note that there is no principled theory to guide the selection of J. The simulation results show that $J = 10$ is a suitable choice.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:07:31 UTC from IEEE Xplore. Restrictions apply.

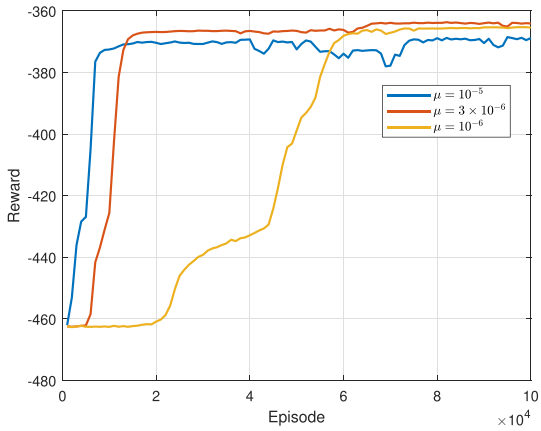


Fig. 4. Convergence of the proposed algorithm on the small timescale (i.e., UAV with fixed position).

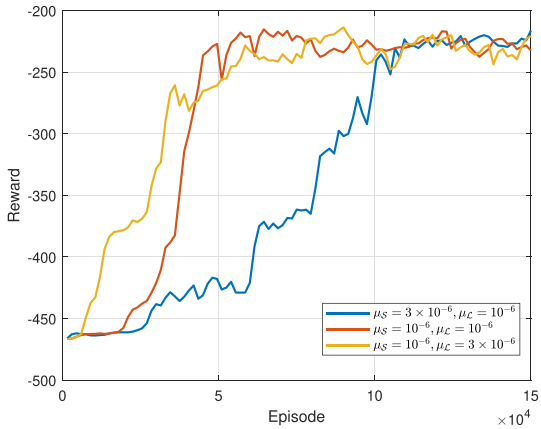


Fig. 5. Convergence of the proposed algorithm with different learning rate pairs.

To validate the performance of the proposed algorithm, we consider the following three baselines:

- *Local Computation Algorithm:* All mobile users perform tasks locally.
- *Fixed Position Algorithm:* The position of the UAV remains fixed and Algorithm 2 is used to optimize the offloading decisions and computing resource allocation.
- *Fixed Action Dimension Algorithm:* The dimension of the output layer of the policy network is not expanded (i.e., $J = 1$).

B. Convergence Performance

We first showcase the convergence of the proposed algorithm on the small timescale by fixing the position of the UAV. As shown in Fig. 4, the best performance is achieved when the learning rate $\mu = 3 \times 10^{-6}$. A smaller learning rate (i.e., 10^{-6}) leads to slower algorithm convergence, while a larger learning rate (i.e., 10^{-5}) results in an unstable training process and poor learning performance.

As shown in Fig. 5, the proposed algorithm converges at all three learning rate pairs, where the first entry μ_S and the second entry μ_L respectively denote the learning rate for the small and large timescales of the proposed algorithm. The best performance is achieved when the learning rate pair is $(10^{-6}, 10^{-6})$. A larger learning rate for the small timescale (i.e., 3×10^{-6}) leads to faster reward growth in the early stages of the training process but results in an unstable training process. In comparison, a larger learning rate for the large timescale leads to slower convergence. This is because a larger learning rate for the large timescale causes the environment to change more rapidly, making it difficult for the algorithm on the small timescale to learn the policy. Note that the optimal learning rate for the small timescale is 10^{-6}, which is smaller than that for the Fixed Position Algorithm in Fig. 4. This is because in the Fixed Position Algorithm, the position of the UAV remains unchanged, which leads to a simpler optimization problem and a larger learning rate is adopted. The fluctuation of the proposed algorithm is greater than that of the Fixed Position Algorithm during the training process because the former has one more timescale, and the sum of the fluctuations of the two timescales results in a greater overall fluctuation.

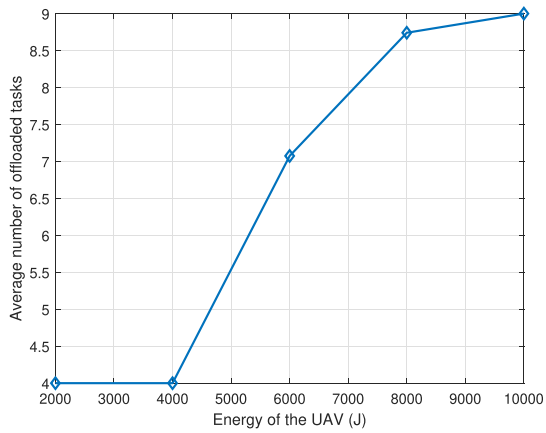


Fig. 6. Average number of offloaded tasks per time slot versus the energy of the UAV for the proposed algorithm.

C. Analysis of Offloading Decisions

We demonstrate the impact of the UAV's energy and bandwidth on the average number of mobile users that offload tasks per time slot (i.e., $\mathbb{E}[M^{\text{Sum}}(l,t)]$) in Figs. 6 and 7, respectively. As shown in Fig. 6, not all users offload tasks. This is due to two main reasons. Firstly, the poor wireless channels result in significant transmission delays, which leads to some users performing tasks locally. Secondly, as the computing resources and energy of the UAV are constrained, the UAV cannot allocate computing resources to all users. Moreover, increasing the UAV's energy enables it to allocate more computing resources, which encourages more users to offload tasks. As shown in Fig. 7, the average number of offloaded tasks increases with increasing bandwidth because larger bandwidth can reduce transmission delay, making users more inclined to offload tasks.

D. Performance Comparison

As shown in Fig. 8, the average delay of the Local Computation Algorithm is not affected by the variation of bandwidth,

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:07:31 UTC from IEEE Xplore. Restrictions apply.

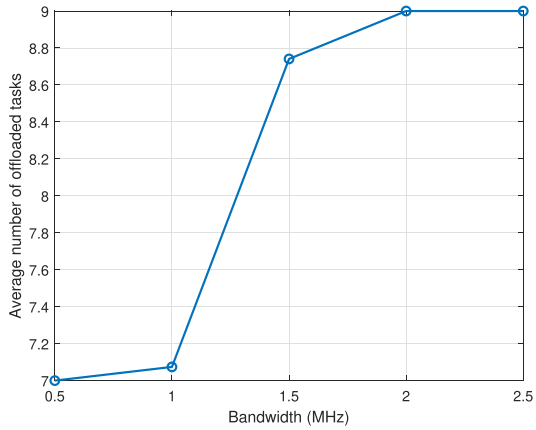


Fig. 7. Average number of offloaded tasks per time slot versus the bandwidth for the proposed algorithm when $E = 6000$.

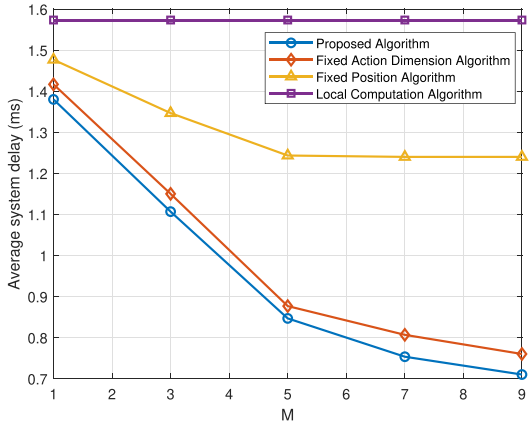


Fig. 9. Average delay versus the number of computing cores of the UAV's CPU.

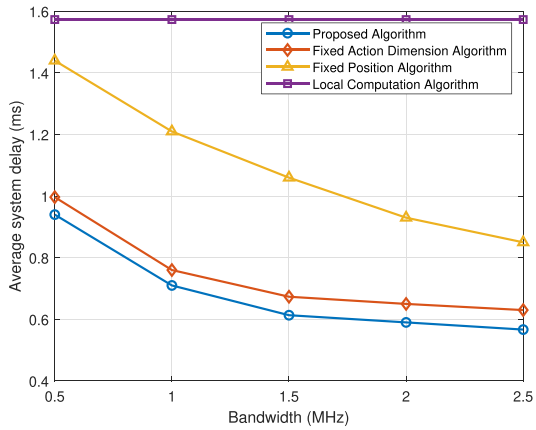


Fig. 8. Average delay versus system bandwidth.

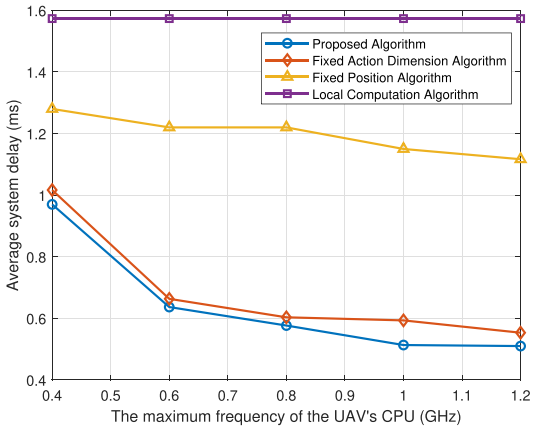


Fig. 10. Average delay versus the maximum frequency of the UAV's CPU with $E = 100,000$.

as it only depends on the users' local computing resources. In addition, the average delay decreases as bandwidth increases for all three other algorithms. This is because increased bandwidth reduces transmission delay, resulting in a decrease in the average delay. The Fixed Position Algorithm has a larger average delay than the proposed algorithm because the former does not optimize the UAV's flight trajectory, which leads to increased transmission delay. The proposed algorithm achieves a lower delay than the Fixed Action Dimension Algorithm due to the inappropriate dimension of the output layer of the policy network of the latter.

Fig. 9 illustrates the relationship between the number of computing cores in the UAV's CPU and the average delay, when $F = 5$ GHz. The average delay of the Fixed Position Algorithm initially decreases and then remains constant as the number of computing cores of the UAV's processor increases. This is because the position of the UAV remains unchanged in the Fixed Position Algorithm, and when the users are far from the UAV, it results in high transmission delay. As a result, some users prefer local computation and do not offload tasks in the Fixed Position Algorithm.

As shown in Fig. 10, the average delay of the proposed algorithm initially decreases and then becomes constant as the maximum UAV CPU frequency increases, when $M = 9$. This is because using more computing resources consumes more energy. In order to better utilize the limited energy, the UAV will not allocate all computing resources to the users. As shown in Fig. 11, the delay of the proposed algorithm, the Fixed Position

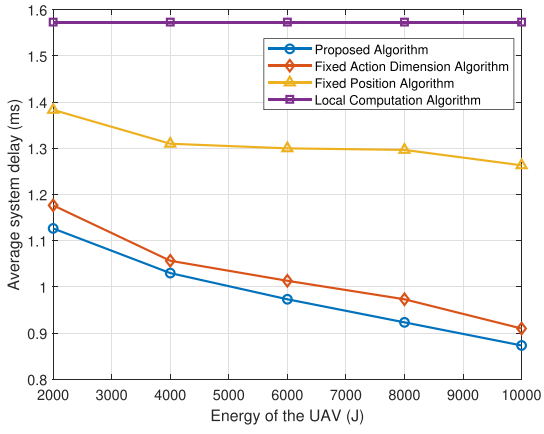


Fig. 11. Average delay versus the energy of the UAV.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:07:31 UTC from IEEE Xplore. Restrictions apply.

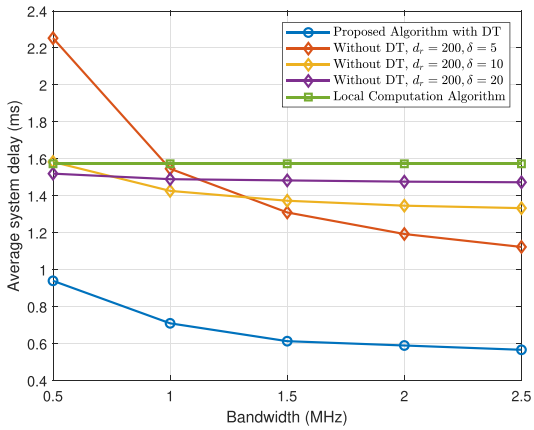


Fig. 12. Average delay versus system bandwidth.

Algorithm, and the No Dimension Spread Algorithm all decrease as the UAV's energy increases. This is because increasing the UAV's energy enables the UAV to allocate more computing resources to users, thereby reducing the computing delay.

When there is no DT in the MEC system, the UAV cannot obtain the real-time system states (i.e., the UAV cannot obtain the time-varying parameters of the system). Therefore, the UAV can only make offloading decisions and allocate computing resources based on the time-invariant parameters of the system. In this case, the UAV first calculates the priority of each user by $\beta_n = \frac{\overline{C}}{f_n} - \frac{\overline{B}}{R_n}$, where $\overline{C}$ is the average required CPU cycles, $\overline{B}$ is the average task size, and R_n is the transmission rate when the distance between the UAV and user n is d_r. Note that β_n is time-invariant and d_r is a hyper-parameter. At each time slot, user n with a larger β_n can prioritize offloading tasks to the UAV, and the UAV allocates $(f_n + \delta)$ CPU cycles to user n, where δ is a hyper-parameter. As shown in Fig. 12, the average delay of the system without DT is higher than that of the system with DT, which verifies the superiority of DT. In addition, increasing δ may not necessarily reduce the average delay, as the energy of the UAV is limited. Specifically, adopting a larger δ quickly depletes the UAV's energy, resulting in the inability to provide computing services for users in subsequent time slots.

VI. Conclusion

In this paper, we proposed a UAV-assisted MEC network with DT, which can guide effective task offloading by leveraging the real-time state information from the physical world. With an objective to minimize the system delay, we developed a customized PPO algorithm, followed by developing a safe projection method to deal with the computing resource constraints. Simulation results show the convergence of the proposed PPO algorithm and demonstrate the superiority of the proposed algorithm compared to the baselines.

References

[1] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 4, pp. 2322–2358, Fourthquarter 2017.

[2] C. You, K. Huang, and H. Chae, "Energy efficient mobile cloud computing powered by wireless energy transfer," *IEEE J. Sel. Areas Commun.*, vol. 34, no. 5, pp. 1757–1771, May 2016.

[3] J. Ren, G. Yu, Y. Cai, and Y. He, "Latency optimization for resource allocation in mobile-edge computation offloading," *IEEE Trans. Wireless Commun.*, vol. 17, no. 8, pp. 5506–5519, Aug. 2018.

[4] W. Fan et al., "Collaborative service placement, task scheduling, and resource allocation for task offloading with edge-cloud cooperation," *IEEE Trans. Mobile Comput.*, vol. 23, no. 1, pp. 238–256, Jan. 2024.

[5] D. Wang, H. Zhu, C. Qiu, Y. Zhou, and J. Lu, "Distributed task offloading in cooperative mobile edge computing networks," *IEEE Trans. Veh. Technol.*, vol. 73, no. 7, pp. 10487–10501, Jul. 2024.

[6] Z. Kuang, L. Li, J. Gao, L. Zhao, and A. Liu, "Partial offloading scheduling and power allocation for mobile edge computing systems," *IEEE Internet Things J.*, vol. 6, no. 4, pp. 6774–6785, Aug. 2019.

[7] Z. Zhao et al., "A novel framework of three-hierarchical offloading optimization for MEC in industrial IoT networks," *IEEE Trans. Ind. Informat.*, vol. 16, no. 8, pp. 5424–5434, Aug. 2020.

[8] L. Wu, P. Sun, H. Chen, Y. Zuo, Y. Zhou, and Y. Yang, "NOMA-enabled multiuser offloading in multicell edge computing networks: A coalition game based approach," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 2, pp. 2170–2181, Mar./Apr. 2024.

[9] Y. Deng, Z. Chen, X. Chen, and Y. Fang, "Task offloading in multi-hop relay-aided multi-access edge computing," *IEEE Trans. Veh. Technol.*, vol. 72, no. 1, pp. 1372–1376, Jan. 2023.

[10] Y. Yang et al., "Multi-agent reinforcement learning based file caching strategy in mobile edge computing," *IEEE/ACM Trans. Netw.*, vol. 31, no. 6, pp. 3159–3174, Dec. 2023.

[11] F. Lu et al., "Resource and trajectory optimization for UAV-relay-assisted secure maritime MEC," *IEEE Trans. Commun.*, vol. 72, no. 3, pp. 1641–1652, Mar. 2024.

[12] P. Mach, Z. Becvar, and M. Nikooroo, "Offloading of tasks with tight delay requirements via combined half and full duplex UAV relays," *IEEE Trans. Veh. Technol.*, vol. 73, no. 9, pp. 14086–14091, Sep. 2024.

[13] X. Li, X. Du, N. Zhao, and X. Wang, "Computing over the sky: Joint UAV trajectory and task offloading scheme based on optimization-embedding multi-agent deep reinforcement learning," *IEEE Trans. Commun.*, vol. 72, no. 3, pp. 1355–1369, Mar. 2024.

[14] C. Zhan, H. Hu, X. Sui, Z. Liu, and D. Niyato, "Completion time and energy optimization in the UAV-enabled mobile-edge computing system," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 7808–7822, Aug. 2020.

[15] R. Dong, C. She, W. Hardjawana, Y. Li, and B. Vucetic, "Deep learning for hybrid 5G services in mobile edge computing systems: Learn from a digital twin," *IEEE Trans. Wireless Commun.*, vol. 18, no. 10, pp. 4692–4707, Oct. 2019.

[16] Z. Yao, S. Xia, Y. Li, and G. Wu, "Cooperative task offloading and service caching for digital twin edge networks: A graph attention multi-agent reinforcement learning approach," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 11, pp. 3401–3413, Nov. 2023.

[17] J. Li et al., "AoI-aware user service satisfaction enhancement in digital twin-empowered edge computing," *IEEE/ACM Trans. Netw.*, vol. 32, no. 2, pp. 1677–1690, Apr. 2024.

[18] B. Li, Y. Liu, L. Tan, H. Pan, and Y. Zhang, "Digital twin assisted task offloading for aerial edge computing and networks," *IEEE Trans. Veh. Technol.*, vol. 71, no. 10, pp. 10863–10877, Oct. 2022.

[19] B. Li, W. Liu, W. Xie, N. Zhang, and Y. Zhang, "Adaptive digital twin for UAV-assisted integrated sensing, communication, and computation networks," *IEEE Trans. Green Commun. Netw.*, vol. 7, no. 4, pp. 1996–2009, Dec. 2023.

[20] V.-P. Bui, S. R. Pandey, P. M. D. S. Ana, and P. Popovski, "Value-based reinforcement learning for digital twins in cloud computing," in *Proc. IEEE Int. Conf. Commun.*, 2024, pp. 1413–1418.

[21] W. Ding, Z. Yang, M. Chen, Y. Liu, and M. Shikh-Bahaei, "Joint vehicle connection and beamforming optimziation in digital twin assisted integrated sensing and communication vehicular networks," *IEEE Internet Things J.*, vol. 11, no. 20, pp. 32923–32938, Oct. 2024.

[22] J. Chen et al., "Deep reinforcement learning based resource allocation in multi-UAV-aided MEC networks," *IEEE Trans. Commun.*, vol. 71, no. 1, pp. 296–309, Jan. 2023.

[23] Q. Luo, T. H. Luan, W. Shi, and P. Fan, "Deep reinforcement learning based computation offloading and trajectory planning for multi-UAV cooperative target search," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 2, pp. 504–520, Feb. 2023.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:07:31 UTC from IEEE Xplore. Restrictions apply.

[24] M. Akbari, A. Syed, W. S. Kennedy, and M. Erol-Kantarci, "Constrained federated learning for AoI-limited SFC in UAV-aided MEC for smart agriculture," *IEEE Trans. Mach. Learn. Commun. Netw.*, vol. 1, pp. 277–295, 2023.
[25] R. Ding, F. Gao, and X. S. Shen, "3D UAV trajectory design and frequency band allocation for energy-efficient and fair communication: A deep reinforcement learning approach," *IEEE Trans. Wireless Commun.*, vol. 19, no. 12, pp. 7796–7809, Dec. 2020.
[26] T. Zhang, K. Zhu, S. Zheng, D. Niyato, and N. C. Luong, "Trajectory design and power control for joint radar and communication enabled multi-UAV cooperative detection systems," *IEEE Trans. Commun.*, vol. 71, no. 1, pp. 158–172, Jan. 2023.
[27] H. Ke, J. Wang, L. Deng, Y. Ge, and H. Wang, "Deep reinforcement learning-based adaptive computation offloading for MEC in heterogeneous vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 69, no. 7, pp. 7916–7929, Jul. 2020.
[28] Z. Shao, H. Yang, L. Xiao, W. Su, Y. Chen, and Z. Xiong, "Deep reinforcement learning-based resource management for UAV-assisted mobile edge computing against jamming," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 13358–13374, Dec. 2024.
[29] J. Lei, T. Zhang, X. Mu, and Y. Liu, "NOMA for STAR-RIS assisted UAV networks," *IEEE Trans. Commun.*, vol. 72, no. 3, pp. 1732–1745, Mar. 2024.
[30] Q. Hou, Y. Cai, Q. Hu, M. Lee, and G. Yu, "Joint resource allocation and trajectory design for multi-UAV systems with moving users: Pointer network and unfolding," *IEEE Trans. Wireless Commun.*, vol. 22, no. 5, pp. 3310–3323, May 2023.
[31] Z. Sun et al., "Tjcct: A two-timescale approach for UAV-assisted mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 24, no. 4, pp. 3130–3147, Apr. 2025.
[32] D. Xu et al., "Tutti: Coupling 5G RAN and mobile edge computing for latency-critical video analytics," in *Proc. 28th Annu. Int. Conf. Mobile Comput. Netw.*, 2022, pp. 729–742.
[33] D. W. Matolak and R. Sun, "Unmanned aircraft systems: Air-ground channel characterization for future applications," *IEEE Veh. Technol. Mag.*, vol. 10, no. 2, pp. 79–85, Jun. 2015.
[34] Y. Chen, Z. Liu, Y. Zhang, Y. Wu, X. Chen, and L. Zhao, "Deep reinforcement learning-based dynamic resource management for mobile edge computing in industrial Internet of Things," *IEEE Trans. Veh. Technol.*, vol. 17, no. 7, pp. 4925–4934, Jul. 2021.
[35] Q. Ren, O. Abbasi, G. K. Kurt, H. Yanikomeroglu, and J. Chen, "Caching and computation offloading in high altitude platform station (HAPS) assisted intelligent transportation systems," *IEEE Trans. Wireless Commun.*, vol. 21, no. 11, pp. 9010–9024, Nov. 2022.
[36] K. Guo, R. Gao, W. Xia, and T. Q. Quek, "Online learning based computation offloading in MEC systems with communication and computation dynamics," *IEEE Trans. Commun.*, vol. 69, no. 2, pp. 1147–1162, Feb. 2021.
[37] A. M. Seid, G. O. Boateng, B. Mareri, G. Sun, and W. Jiang, "Multi-agent DRL for task offloading and resource allocation in multi-UAV enabled IoT edge network," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 4, pp. 4531–4547, Dec. 2021.
[38] Y. Mao, J. Zhang, and K. B. Letaief, "Dynamic computation offloading for mobile-edge computing with energy harvesting devices," *IEEE J. Sel. Areas Commun.*, vol. 34, no. 12, pp. 3590–3605, Dec. 2016.
[39] L. T. Hoang, C. T. Nguyen, and A. T. Pham, "Deep reinforcement learning-based online resource management for UAV-assisted edge computing with dual connectivity," *IEEE/ACM Trans. Netw.*, vol. 31, no. 6, pp. 2761–2776, Dec. 2023.
[40] J. Ji, K. Zhu, and L. Cai, "Trajectory and communication design for cache-enabled UAVs in cellular networks: A deep reinforcement learning approach," *IEEE Trans. Mobile Comput.*, vol. 22, no. 10, pp. 6190–6204, Oct. 2023.
[41] J. Clifton and E. Laber, "Q-learning: Theory and applications," *Annu. Rev. Statist. Appl.*, vol. 7, pp. 279–301, 2020.
[42] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," 2015, *arXiv:1509.02971*.
[43] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
[44] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2018.
[45] M. Holzleitner, L. Gruber, J. Arjona-Medina, J. Brandstetter, and S. Hochreiter, "Convergence proof for actor-critic methods applied to PPO and RUDDER," in *Proc. Trans. Large-Scale Data-And Knowledge-Centered Syst. XLVIII: Special Issue Memory Univ. Prof. Dr. Roland Wagner*. Springer, 2021, pp. 105–130.
[46] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*. Cambridge, MA, USA: MIT Press, 2022.
[47] S. Batabyal and P. Bhaumik, "Mobility models, traces and impact of mobility on opportunistic routing algorithms: A survey," *IEEE Commun. Surveys Tuts.*, vol. 17, no. 3, pp. 1679–1707, Thirdquarter 2015.



Luteng Qiao (Member, IEEE) received the B.S. degree in telecommunications engineering from Xidian University, Xi'an, China, in 2021. He is currently working toward the Ph.D. degree with the School of Information Science and Technology, ShanghaiTech University, Shanghai, China. His research interests include the Internet of Things, edge computing, and age of information.



Yong Zhou (Senior Member, IEEE) received the B.Sc. and M.Eng. degrees from Shandong University, Jinan, China, in 2008 and 2011, respectively, and the Ph.D. degree from the University of Waterloo, Waterloo, ON, Canada, in 2015. From 2015 to 2018, he was a Postdoctoral Researcher Fellow in the Department of Electrical and Computer Engineering, The University of British Columbia, Vancouver, BC, Canada. Since 2018, he has been with the School of Information Science and Technology, ShanghaiTech University, Shanghai, China, where he is currently a Tenured Associate Professor. His research interests include 6 G communications, edge intelligence, and Internet of Things. He is an Associate Editor for IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS and IEEE Open Journal of the Communications Society. He was the Track Co-Chair of IEEE VTC 2020 Fall and IEEE VTC 2023 Spring, and the Co-Chair of IEEE ICC 2022 workshop on edge artificial intelligence for 6G and IEEE Globecom 2024 workshop on space computing power networks.



Yuanming Shi (Senior Member, IEEE) received the B.S. degree in electronic engineering from Tsinghua University, Beijing, China, in 2011, and the Ph.D. degree in electronic and computer engineering from The Hong Kong University of Science and Technology, Hong Kong, in 2015. Since 2015, he has been with the School of Information Science and Technology in ShanghaiTech University, Shanghai, China, where he is currently a Full Professor. His research interests include edge artificial intelligence and large-scale optimization. He was the recipient of the IEEE Marconi Prize Paper Award in Wireless Communications in 2016, Young Author Best Paper Award by IEEE Signal Processing Society in 2016, IEEE ComSoc Asia-Pacific Outstanding Young Researcher Award in 2021, Chinese Institute of Electronics First Prize in Natural Science in 2022, and China Institute of Communications First Prize in Natural Science in 2024. He is also an Editor of IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS, IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, *Journal of Communications and Information Networks*, and *Space Habitation*. He is an IET Fellow.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:07:31 UTC from IEEE Xplore. Restrictions apply.



Sheng Wu received the B.S. and M.S. degrees from the Beijing University of Posts and Telecommunications, Beijing, China, in 2004 and 2007, respectively, and the Ph.D. degree in electronic engineering from Tsinghua University, Beijing, in 2014. He was a Postdoctoral Researcher with the Tsinghua Space Center, Tsinghua University. He is currently a Professor with the Beijing University of Posts and Telecommunications. He has authored or coauthored more than 100 journals and conference papers, such as IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS, IEEE TRANSACTIONS ON COMMUNICATIONS. He also holds more than 30 granted patents. His research interests include iterative detection and decoding, channel estimation, massive MIMO, and satellite communications. He was the recipient of the First Prize from the Science and Technology Award of Chinese Institute of Electronics in 2017, Silver Medal from the 46th Geneva International Exhibition of Inventions in 2018, and Second Prize from the National Technological Invention Award of China in 2018, and Natural Science Foundation of China Excellent Young Scientists Fund Award in 2020.



Tom H. Luan (Senior Member, IEEE) received his B.E. degree in electrical and computer engineering from Xi'an Jiaotong University, Xi'an, China, the M.S. degree in electrical and computer engineering from the Hong Kong University of Science and Technology, Hong Kong, and the Ph.D. degree in electrical and computer engineering from the University of Waterloo, Waterloo, ON, Canada. From 2013 to 2017, he was a Lecturer in Mobile and Apps with Deakin University, Geelong, VIC, Australia. From 2017 to 2022, he was a Professor in the School of Cyber Engineering with Xidian University, Xi'an. Since 2022, he has been a Professor in the School of Cyber Science and Engineering at Xi'an Jiaotong University. He has authored more than 200 peer-reviewed papers in top journals and conferences, such as IEEE/ACM TRANSACTIONS ON NETWORKING, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON MULTIMEDIA, IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY, and IEEE INFOCOM. His research interests include the optimization and protocol design of the Metaverse, digital twin networks, AI-agent networks, and the security and performance evaluation of vehicular networks and edge computing systems. Dr. Luan's was the recipient of the IEEE VTS Best Land Transportation Paper Award in 2017 and Best Paper Award at IEEE VTC 2023.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:07:31 UTC from IEEE Xplore. Restrictions apply.